

Sichere Software

X – ÜB – Angriffe auf die Seitentabelle

Aufgabe 1: Grundkonzepte

- a) Welchen konzeptionellen Vorteil bringt die Aufteilung der CPU in ein Steuerwerk und ein Rechenwerk?
- b) Wie gewährleistet der Speicherschutz mittels Seitentabelle und virtuellen Adressen, dass ein Prozess nur auf eigene Adressen zugreift?
- c) Zum Dekodieren als Teilschritt der Befehlsausführung
- Warum ist es nötig, Maschinencodebefehle zu dekodieren?
 - Was genau passiert beim Dekodieren eines Befehls?
 - Warum kann man den Befehlszähler ohne Dekodieren eventuell nicht korrekt zum nächsten Befehl inkrementieren?
- d) Der *Speedup* ist der Wert $\frac{T_1}{T_p}$, wobei T_1 die Ausführungsdauer bei komplett serieller Abarbeitung und T_p die Ausführungsdauer bei paralleler Verarbeitung darstellt.
- Welcher Speedup lässt sich mit einer n -stufigen Pipeline erzielen?
 - Wie groß ist der maximal erreichbare Speedup, wenn man beliebig viele Stufen in einer Pipeline vorsehen kann?
 - Warum ist dieser maximale Speedup in der Praxis nicht erreichbar? Geben Sie zwei konzeptionell unterschiedliche Argumente an.
- e) Welchen Einfluss hat die Out-Of-Order-Execution auf den Speedup?
- f) Warum lohnt es sich nicht, für jeden Sprungbefehl in einem Programm dessen Verhalten in einem Register der CPU nachzuvollziehen?

Aufgabe 2: Programmierpraxis

- a) Schreiben Sie ein C-Programmstück, das ein Write-After-Write-Hazard beinhaltet. Erklären Sie, warum hier überhaupt ein Hazard vorliegt: Man könnte ja argumentieren, dass der zweite Schreibzugriff ohnehin das Datum vom ersten Schreibzugriff überschreibt. Wo liegt der Denkfehler?
- b) Schreiben Sie ein C-Programm, mit dem Sie den Speedup durch Pipelining nachmessen können. Achten Sie dabei auf folgende Punkte:
- Jede moderne CPU nimmt standardmäßig Pipelining vor. Zum Gegenprüfen müssen Sie also verhindern, dass ihre CPU das Pipelining durchführen kann.

Sie brauchen dazu ein Pipeline-Hazard – allerdings derart, dass sowohl mit Hazard wie ohne Hazard die selben Maschinencodebefehle mit (weitgehend) den selben Daten durchlaufen.

- Mit einem geeigneten Pipeline Hazard können Sie auch die Out-Of-Order-Execution verhindern – nämlich, wenn das Pipeline Hazard jeden weiteren Befehl in einem gemessenen Programmstück betrifft.
- Messen Sie im Debug- und im Release-Modus.
- **Achtung:** Compiler sind schlau. Compiler können selbständig Instruktionen umsortieren. Achten Sie darauf, dass Ihr Compiler nicht vorher schon den Maschinencode optimiert (z. B. indem Sie die Eingabewerte erst zur Laufzeit mittels Zufallszahlengenerator erzeugen). Prüfen Sie den Maschinencode im Assembly gegen, um sicher zu gehen.

c) Gehen Sie vor wie im vorangehenden Teilpunkt, aber für die Out-Of-Order-Ausführung, die ebenfalls in allen modernen CPUs integriert ist.

d) Es ist auf den ersten Blick nicht ganz ersichtlich, wie aus den Optimierungen der Vorlesung ein Sicherheitsproblem entsteht. Dies ist im Folgenden zu prüfen.

- Angenommen, Sie haben eine Funktion `get_data(int index)` wie folgt gegeben (z. B. als Teil eines Java-Skript-Interpreters). Sie können die Funktion mit beliebigem Parameter `index` aufrufen.

```
// ein beliebiges Hilfsarray, auf das Sie keinen direkten Zugriff  
// in Ihrem Programm besitzen  
const size_t size_ = 128;  
char array[size_] =  
    { /** ... unwichtige Daten, z. B. Text einer Webseite ... */ };  
  
char get_data(size_t index) // aufrufbare Funktion  
{  
    if (index >= size_) return '\\0';  
    return array[index];  
}
```

Gibt es einen Parameter `index`, den Sie in die Funktion geben können, so dass undefiniertes Verhalten auftritt?

- Angenommen, Sie haben eine CPU, die Sprungvorhersagen nutzt, um die Pipeline zu füllen. Wie können Sie beeinflussen, welche Sprungvorhersage die Prüfung in der Funktion `get_data()` macht, wenn Sie die Funktion nur mit beliebigem `index` aufrufen können?
- Wie bekommen Sie die Funktion `get_data()` mittels Sprungvorhersage und geeignetem Aufruf dazu, auf eine beliebige, bekannte Speicherstelle (z. B. gegeben als Zeiger `char* ptr`) zuzugreifen?
- Warum darf der obige Zugriff nach Vorhersage auf eine geschützte Speicherstelle keine negativen Konsequenzen (z. B. Programmabbruch) haben? Sie können zum Beispiel mit einem Zugriff auf einen `char` mit der Adresse 0 argumentieren. Welcher `index` ist dazu nötig?

- Angenommen, die Sprungvorhersage hat sich geirrt. Warum hilft Ihnen die Rückgabe der Funktion in dem Fall nicht, den Wert der gewählten Speicherzelle direkt auszulesen?
- Was muss die CPU rückgängig machen, wenn die Sprungvorhersage sich geirrt hat – konkret im Fall des Speicherzugriffs? **Hinweis:** Berücksichtigen Sie hier die Speicherarchitektur (z. B. aus Betriebssysteme).
- Warum fällt es der CPU schwer, den Zugriff komplett rückgängig zu machen? Anders formuliert: Welcher Seiteneffekt eines Speicherzugriffs kann eine größere Menge an Rechenplatz und Speicherplatz erfordern, um sich den früheren Zustand zu merken?
- Wie könnte man den Seiteneffekt ausnutzen, um Informationen über das Ergebnis von `array[index]` für einen **beliebigen** Index aus der Sprungvorhersage trotz Rückgängigmachen auszuleiten? Sie müssen dazu mit der Rückgabe der Funktion `get_data(...)` geeignet weiterrechnen – der Seiteneffekt, den `array[index]` verursacht, ist nicht aufschlussreich genug.
- Wie könnten Sie dem Seiteneffekt durch Messung zuleibe rücken, um den Wert von `array[index]` für den beliebigen Index tatsächlich zu ermitteln?

e) Lesen Sie die beiden Veröffentlichungen `meltdown.pdf` und `spectre.pdf` (siehe OPAL). Beantworten Sie dann folgende Fragen:

- Geben Sie in eigenen Worten wieder, wie jeder der Angriffe funktioniert.
- Welche Schwachstelle wird jeweils ausgenutzt? Was für Voraussetzungen an die Hardware sind jeweils nötig?
- Wie geht ein Angreifer vor, um die Schwachstelle auszunutzen? Welche Art von Zugriff braucht er auf den Computer?
- Was für Daten kann der Angreifer jeweils ausleiten?
- Was sind Gemeinsamkeiten bzw. Unterschiede der Sicherheitslücken? (vgl. die Erkenntnisse der vorangehenden Punkte)
- Warum ist es schwierig, die Sicherheitslücken zu beheben? Welche grundsätzlichen Möglichkeiten bestehen – durch Anpassung der Software, der Hardware oder des Betriebssystems?
- Welches der beiden Paper beschreibt die im vorigen Teilpunkt ausgenutzte Sicherheitslücke?

f) Zu der Spectre-Lücke im Speziellen gibt es alternative Angriffsimplementierungen. Recherchieren Sie selbständig, wie ein aktueller Spectre-Angriff funktionieren könnte, und geben Sie den Angriff in eigenen Worten wieder.